Hochschule Konstanz
Fakultät Informatik

Software Architecture

ParserCombinators

Prof. Dr. Marko Boger

Internal vs External DSLs

Internal DSLs are embedded into a host language. They can be used within a programming language or environment. IDEs support editing internal DSLs and their execution.

External DSLs are independent of any language environment. They can be read in from a text file or a command line. They are executed by interpreting the input. Editors need to specifically developed for it, if support is wanted.

External DSLs have their own grammar.

They are read by parsers.

Parsers in Scala

In Scala, a parser can be developed using a library called Parser Combinators.

A combinator is a higher-order function that uses only function application and earlier defined combinators to define a result from its arguments.

Scala Parser Combinators is an internal DSL for writing parsers.

The Goal: A DSL for Bets

Tipp-Abgabe: Fussball WM 2018

Frage: Wer spielt im Finale?

Wähle 2 aus

- Ägypten > 6.5
- Argentinien > 4.5
- Australien > 10
- Belgien > 3
- Brasilien > 1.5
- Costa Rica > 7.5
- Dänemark > 5
- Deutschland > 1.5
- England > 3.5

...

FIFA WORLD CUP RUSSIA 2018

https://images.sport1.de/imagix/svg/3da0b918-4ccd-11e6-a9b7-f80f41fc6a62

The DSL in Backus-Naur form

```
<syntax> ::= <bet> <question> <choiceNumber> <repChoice>
<bet> ::= "Tipp-Abgabe" <colon> <stringLiteral>
<question> ::= "Frage" <colon> <stringLiteral>
<colon> ::= ":"
<choiceNumber> ::= "Wähle" <optInt> "aus"
<optInt> ::= <int> | ""
<repChoice> ::= <choice> <repChoice> | <choice>
<choice> ::= <singleChoice> | <multiChoice>
<singleChoice> ::= <choiceRest>
<multiChoice> ::= <int> ".." <int> <choiceRest>
<choiceRest> ::= ">" <int>
<int> ::= <digit> | <digit> <int>
<digit> ::= "0..9"
```

The same DSL in Scala Parser Combinators

```
private def topicParser: Parser[Topic] =
 "Tipp-Abgabe:" ~ text ~
 "Frage:" ~ text ~
 "Wähle" ~ count ~ "aus" ~ choices ^^ {
   case _ ~ t ~ _ ~ n ~ _ ~ c ~ _ ~ ch =>
      Topic(t, n, c, ch)
 }
```

Tipp-Abgabe: Fussball WM 2018
Frage: Wer spielt im Finale?
Wähle 2 **aus**
Ägypten **>** 6.5
Argentinien **>** 4.5
...

Scala Parser Combinators

Scala Parser Combinators is a Scala internal DSL to make the development of Parsers easier.

- Github: scala/scala-parser-combinators
- until Scala 2.10 as part of the scala api
- community-maintained since Scala 2.11

```
libraryDependencies +=
  "org.scala-lang.modules" %%
    "scala-parser-combinators" % "2.1.1"
```

Combinators: The operations of this DSL

Combinators:

```
parser ~ parser      literal ~> parser      parser <~ literal
```

No Backtracking:

```
parser ~! parser     literal ~>! parser     parser <~! literal
```

Optional:

```
?        opt(parser)
```

Alternative:

```
| // first match      ||| // longest match
```

Repeat:

```
*        rep(parser)
+        rep1(parser)   // min 1 times
         repN(times, parser)
         repsep(parser, separator) // with separator
```

Doing something with the Result

Function application:

```
parser ~ literal ^^ {case p ~ _ => ???}
parser <~ literal ^^ (p => ???)
parser ^^^ 2 // return value on match
parser into {p => parserWithArg(p)}
parser >> {p => parserWithArg(p)}
```

Regex:

```
""".+\d\s+""".r ^^ (_.trim)
".+\\d\\s+".r ^^ (_.trim)
regex(".+\\d\\s+".r) ^^ (_.trim)
```

Example for RegEx Parser

Useful simple parsers:

```
// text without line breaks
"""[^\v]+""".r ^^ (_.trim)

// integer
"""\d+""".r ^^ (_.toInt)

// double
"""\d+(\.\d*)?""".r ^^ (_.toDouble)

// line break
"""\v+""".r ^^^ (():Unit)
```

Bet Game Example

FIFA World Cup 2018

FIFA WORLD CUP
RUSSIA 2018

https://images.sport1.de/imagix/svg/3da0b918-4ccd-11e6-a9b7-f80f41fc6a62

Bet Game Example

FIFA World Cup 2018

- topic
- n choices
- payout factor
- stake

WM 2018 - Gruppenspiele Wettarten einblenden

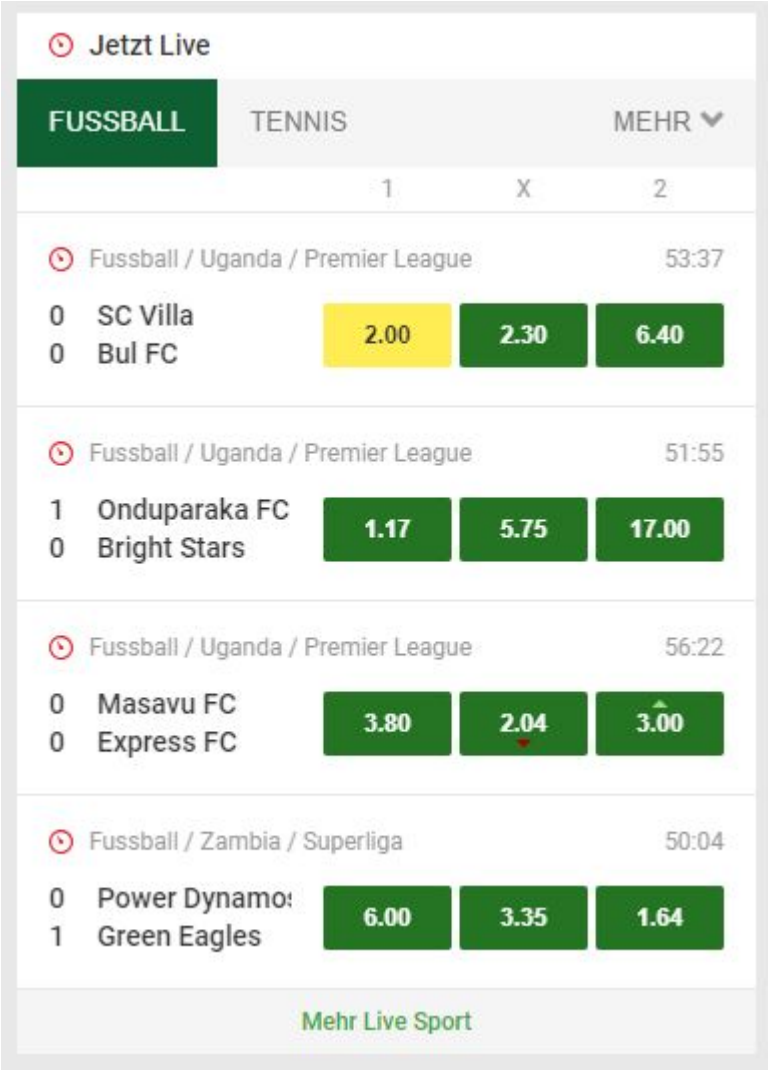
Gruppe A			1	X	2		
Donnerstag, 14.06.							
17:00	Russland	Saudi Arabien	1,32	4,8	12	LIVE	+46
Freitag, 15.06.							
14:00	Ägypten	Uruguay	5,3	3,6	1,7	LIVE	+46
Dienstag, 19.06.							
20:00	Russland	Ägypten	2,0	3,2	4,2	LIVE	+46
Mittwoch, 20.06.							
17:00	Uruguay	Saudi Arabien	1,25	5,7	13	LIVE	+46
Montag, 25.06.							
16:00	Saudi Arabien	Ägypten	5,5	3,6	1,7	LIVE	+5
16:00	Uruguay	Russland	2,45	3,2	3,0	LIVE	+5

Gruppe B			1	X	2		
Freitag, 15.06.							
17:00	Marokko	Iran	2,3	3,0	3,5	LIVE	+46
20:00	Portugal	Spanien	4,3	3,4	1,9	LIVE	+46
Mittwoch, 20.06.							
14:00	Portugal	Marokko	1,55	4,0	6,5	LIVE	+46
20:00	Iran	Spanien	20	6,5	1,18	LIVE	+46
Montag, 25.06.							
20:00	Iran	Portugal	8,5	5,0	1,37	LIVE	+5
20:00	Spanien	Marokko	1,37	4,8	9,0	LIVE	+5

Bet Game Example

FIFA World Cup 2018

- topic
- n choices
- payout factor
- stake



Bet Game Example

FIFA World Cup 2018

- topic
- n choices
- payout factor
- stake

Fußball | Tennis | Basketball | Volleyball | Weitere Sportarten

WM 2018-Spiele / Spiel								
17.06.2018								
17:00	Deutschland	1,50	X	4,00	Mexiko	7,50		+104 ›
23.06.2018								
20:00	Deutschland	1,50	X	4,20	Schweden	6,50		+7 ›
27.06.2018								
16:00	Südkorea	11,00	X	6,00	Deutschland	1,25		+7 ›

Bet Game Example

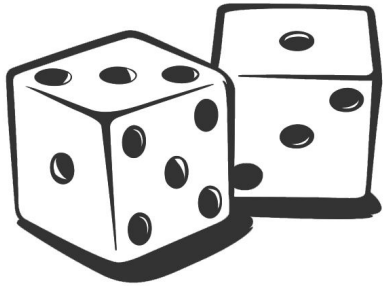
Bet Game Model

```
case class Topic(
    eventName: String,
    topicName: String,
    choiceCountPerPrediction: Int,
    choices: List[Choice]
)

case class Choice(
    id: Int,
    choiceName: String,
    payoutFactor: Double
)
```

Bet Game Example

https://akns-images.eonline.com/eol_images/Entire_Site/2018419/rs_1024x759-180519053454-1024-Royal-Kiss-Royal-Wedding-J111R-051918.jpg?fit=inside|900:auto&output-quality=90



https://www.auto-aufkleber.de/images/product_images/original_images/710_0.gif

https://3c1703fe8d.site.internapcdn.net/newman/gfx/news/hires/2016/canmathshelp.jpg

Bet Game DSL

Tipp-Abgabe: Fussball WM 2018

Frage: Wer spielt im Finale?

Wähle 2 aus

- Ägypten > 6.5
- Argentinien > 4.5
- Australien > 10
- Belgien > 3
- Brasilien > 1.5
- Costa Rica > 7.5
- Dänemark > 5
- Deutschland > 1.5
- England > 3.5

...

FIFA WORLD CUP
RUSSIA 2018

https://images.sport1.de/imagix/svg/3da0b918-4ccd-11e6-a9b7-f80f41fc6a62

Backus-Naur form

```
<syntax> ::= <bet> <question> <choiceNumber> <repChoice>
<bet> ::= "Tipp-Abgabe" <colon> <stringLiteral>
<question> ::= "Frage" <colon> <stringLiteral>
<colon> ::= ":"
<choiceNumber> ::= "Wähle" <optInt> "aus"
<optInt> ::= <int> | ""
<repChoice> ::= <choice> <repChoice> | <choice>
<choice> ::= <singleChoice> | <multiChoice>
<singleChoice> ::= <choiceRest>
<multiChoice> ::= <int> ".." <int> <choiceRest>
<choiceRest> ::= ">" <int>
<int> ::= <digit> | <digit> <int>
<digit> ::= "0..9"
```

Bet Game DSL

Tipp-Abgabe: Royal Wedding 2018

Frage: Wie lange dauert der erste Kuss von Prinz Harry und Meghan Markle?

Wähle aus

- 0.2 Sekunden > 2.1
- 0.3 Sekunden > 2
- 0.4 Sekunden > 1.9
- 0.5 Sekunden > 1.8
- 0.6 Sekunden > 1.7
- 0.7 Sekunden > 1.5
- 0.8 Sekunden > 1.5
- 0.9 Sekunden > 1.7

...

count = 1

https://akns-images.eonline.com/eol_images/Entire_Site/2018419/rs_1024x759-180519053454-1024-Royal-Kiss-Royal-Wedding-J111R-051918.jpg?fit=inside|900:auto&output-quality=90

Bet Game DSL

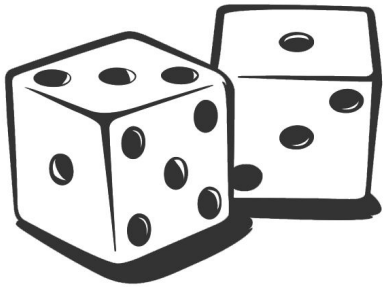
Tipp-Abgabe: Würfelwurf mit zwei Würfeln

Frage: Wie viele Augen sind zu sehen?

Wähle 3 **aus**

- 2 Augen **>** 4
- 3 Augen **>** 3.5
- 4 Augen **>** 3
- 5 Augen **>** 2.5
- 6 Augen **>** 2
- 7 Augen **>** 1.5
- 8 Augen **>** 2
- 9 Augen **>** 2.5
- 10 Augen **>** 3

...



https://www.auto-aufkleber.de/images/product_images/original_images/710_0.gif

Bet Game DSL

Tipp-Abgabe: Casino Roulette

Frage: Welche Zahl trifft die Kugel?

Wähle aus

1 **..** 35 **>** 36

0 **>** 0

multiple choices
1-35 in single line

https://3c1703fe8d.site.internapcdn.net/newman/gfx/news/hires/2016/canmathshelp.jpg

The Model: This is the Data Structure we want to fill

```
case class Topic(
    eventName: String,
    topicName: String,
    choiceCountPerPrediction: Int,
    choices: List[Choice]
)

case class Choice(
    id: Int,
    choiceName: String,
    payoutFactor: Double
)
```

We can build a full blown Parser

```
class ExampleParser extends Parsers {

  // ...
  // ...
  // ...

}
```

Or a simpler RegEx Parser

```
class ExampleParser extends RegexParsers {

  // ...
  // ...
  // ...

}
```

Scala Parser Combinators
~ “followed by”

```
private def topicParser: Parser[Topic] =
 "Tipp-Abgabe:" ~ text ~
 "Frage:" ~ text ~
 "Wähle" ~ count ~ "aus" ~ choices ^^ {
   case _ ~ t ~ _ ~ n ~ _ ~ c ~ _ ~ ch =>
      Topic(t, n, c, ch)
 }
```

Tipp-Abgabe: Fussball WM 2018
Frage: Wer spielt im Finale?
Wähle 2 **aus**
Ägypten **>** 6.5
Argentinien **>** 4.5
...

Scala Parser Combinators
The Grammar

```
private def topicParser: Parser[Topic] =
  "Tipp-Abgabe:" ~ text ~
  "Frage:" ~ text ~
  "Wähle" ~ count ~ "aus" ~ choices ^^ {
    case _ ~ t ~ _ ~ n ~ _ ~ c ~ _ ~ ch =>
        Topic(t, n, c, ch)
  }
```

Tipp-Abgabe: Fussball WM 2018
Frage: Wer spielt im Finale?
Wähle 2 **aus**
Ägypten **>** 6.5
Argentinien **>** 4.5
...

Scala Parser Combinators
The Return Type

```
private def topicParser: Parser[Topic] =
  "Tipp-Abgabe:" ~ text ~
  "Frage:" ~ text ~
  "Wähle" ~ count ~ "aus" ~ choices ^^ {
    case _ ~ t ~ _ ~ n ~ _ ~ c ~ _ ~ ch =>
        Topic(t, n, c, ch)
  }
```

Tipp-Abgabe: Fussball WM 2018
Frage: Wer spielt im Finale?
Wähle 2 **aus**
Ägypten **>** 6.5
Argentinien **>** 4.5
...

Scala Parser Combinators
Constructing the Return Type

```
private def topicParser: Parser[Topic] =
  "Tipp-Abgabe:" ~ text ~
  "Frage:" ~ text ~
  "Wähle" ~ count ~ "aus" ~ choices ^^ {
    case _ ~ t ~ _ ~ n ~ _ ~ c ~ _ ~ ch =>
      Topic(t, n, c, ch)
  }
```

Tipp-Abgabe: Fussball WM 2018
Frage: Wer spielt im Finale?
Wähle 2 **aus**
Ägypten **>** 6.5
Argentinien **>** 4.5
...

Scala Parser Combinators
Deeper Parsers

```
private def topicParser: Parser[Topic] =
  "Tipp-Abgabe:" ~ text ~
  "Frage:" ~ text ~
  "Wähle" ~ count ~ "aus" ~ choices ^^ {
    case _ ~ t ~ _ ~ n ~ _ ~ c ~ _ ~ ch =>
        Topic(t, n, c, ch)
  }
```

Tipp-Abgabe: Fussball WM 2018
Frage: Wer spielt im Finale?
Wähle 2 **aus**
Ägypten **>** 6.5
Argentinien **>** 4.5
...

Scala Parser Combinators

```
// every char until line break
private def text: Parser[String] =
  """[^\v]+""".r ^^ (_.toString)
```

everything but
no line-breaks

Tipp-Abgabe: Fussball WM 2018
Frage: Wer spielt im Finale?
Wähle 2 **aus**
Ägypten **>** 6.5
Argentinien **>** 4.5
...

Scala Parser Combinators

```
private def topicParser: Parser[Topic] =
 "Tipp-Abgabe:" ~ text ~
 "Frage:" ~ text ~
 "Wähle" ~ count ~ "aus" ~ choices ^^ {
   case _ ~ t ~ _ ~ n ~ _ ~ c ~ _ ~ ch =>
      Topic(t, n, c, ch)
 }
```

Tipp-Abgabe: Fussball WM 2018
Frage: Wer spielt im Finale?
Wähle 2 **aus**
Ägypten **>** 6.5
Argentinien **>** 4.5
...

Scala Parser Combinators

```
private def integer: Parser[Int] =
  """\d+""".r ^^ (_.toInt)

// optional integer with default value
private def count: Parser[Int] =
  opt(integer) ^^ (_.getOrElse(1))
```

Tipp-Abgabe: Fussball WM 2018
Frage: Wer spielt im Finale?
Wähle 2 aus
Ägypten > 6.5
Argentinien > 4.5
...

Tipp-Abgabe: Royal Wedding 2018
Frage: Länge erster Kuss von Prinz Harry und Meghan Markle?
Wähle aus
0.2 Sekunden > 2.1
0.3 Sekunden > 2
...

Scala Parser Combinators

```
private def topicParser: Parser[Topic] =
 "Tipp-Abgabe:" ~ text ~
 "Frage:" ~ text ~
 "Wähle" ~ count ~ "aus" ~ choices ^^ {
   case _ ~ t ~ _ ~ n ~ _ ~ c ~ _ ~ ch =>
      Topic(t, n, c, ch)
 }
```

Tipp-Abgabe: Fussball WM 2018
Frage: Wer spielt im Finale?
Wähle 2 **aus**
Ägypten **>** 6.5
Argentinien **>** 4.5
...

Scala Parser Combinators

Tipp-Abgabe: Casino Roulette
Frage: Welche Zahl trifft die Kugel?
Wähle aus

1 > 36.0
2 > 36.0
3 > 36.0
4 > 36.0
...
0 > 0

Tipp-Abgabe: Casino Roulette
Frage: Welche Zahl trifft die Kugel?
Wähle aus

1 .. 35 > 36
0 > 0

Scala Parser Combinators

```
// important order: check first, whether this is a
// multiNumberChoice and then simpleChoice to
// allow points in choice name
private def choices: Parser[List[Choice]] =
  rep(multiNumberChoice | simpleChoice) ^^
    (c => c.flatten)
```

Tipp-Abgabe: Fussball WM 2018

Frage: Wer spielt im Finale?

Wähle 2 **aus**

Ägypten **>** 6.5

Argentinien **>** 4.5

...

Scala Parser Combinators

```
private def choiceName: Parser[String] =
  """[^\v>]+""".r ^^ (_.trim)

private def double: Parser[Double] =
 """\d+(\.\d+)?""".r ^^ (_.toDouble)
```

Ägypten > 6.5

Scala Parser Combinators

```
private def choiceName: Parser[String] =
  """[^\v>]+""".r ^^ (_.trim)

private def simpleChoice: Parser[List[Choice]] =
  choiceName ~ ">" ~ double ^^ {
    case t ~ _ ~ d => List(Choice(t, d))
  }
```

Tipp-Abgabe: Fussball WM 2018

Frage: Wer spielt im Finale?

Wähle 2 **aus**

Ägypten > 6.5

Argentinien > 4.5

...

Scala Parser Combinators

Tipp-Abgabe: Casino Roulette

Frage: Welche Zahl trifft die Kugel?

Wähle aus

1 .. 35 > 35.0

0 > 0

```
private def multiNumberChoice: Parser[List[Choice]] =
  integer ~ ".." ~ integer ~ ">" ~ double ^^ {
    case from ~ _ ~ to ~ _ ~ f =>
      from.to(to).map(n => Choice(n.toString, f)).toList
 }
```

Scala Parser Combinators

```
// important order: check first, whether this is a
// multiNumberChoice and then simpleChoice to
// allow points in choice name
private def choices: Parser[List[Choice]] =
  rep(multiNumberChoice | simpleChoice) ^^
    (c => c.flatten)
```

Tipp-Abgabe: Fussball WM 2018

Frage: Wer spielt im Finale?

Wähle 2 **aus**

Ägypten **>** 6.5

Argentinien **>** 4.5

...

The Parser Function
it now needs a class

```
private def topicParser: Parser[Topic] =
 "Tipp-Abgabe:" ~ text ~
 "Frage:" ~ text ~
 "Wähle" ~ count ~ "aus" ~ choices ^^ {
   case _ ~ t ~ _ ~ n ~ _ ~ c ~ _ ~ ch =>
      Topic(t, n, c, ch)
 }
```

The surrounding Class "The Parser"

```
class ExampleParser extends RegexParsers {

  def parseDSL(input: String): ParseResult[Topic] =
    parse(topicParser, input)

  // no input left over
  def parseDSL(input: String): ParseResult[Topic] =
    parseAll(topicParser, input)

  // ...
}
```

Scala Parser Combinators

```
class ExampleParser extends RegexParsers {

  def parseDSL(input: String):
      Either[String, Topic] =
    parseAll(topicParser, input) match {
      case Success(t, _) => Right(t)
      case NoSuccess(msg, next) =>
        val pos = next.pos
        Left(s"[$pos] failed parsing: $msg
                                  \n\n${pos.longString})")
    }
  // ...
}
```

Transform

```
object TopicTransformer {

  def apply(parseResult: Either[String, Topic]):
      Either[String, Topic] =
    for {
      parsed <- parseResult
      numbers <- checkChoiceNumber(parsed)
      choices <- checkDuplicateChoices(numbers)
    } yield {
      choices
    }
}
```

Transform

```
private def checkChoiceNumber(topic: Topic):
  Either[String, Topic] =

 if (topic.choiceCountPerPrediction >= topic.choices.size) {
   Left("There are not enough choices!")
 } else {
   Right(topic)
 }
```

Transform

```
private def checkDuplicateChoices(topic: Topic):
  Either[String, Topic] =

 if (topic.choices.map(_.choiceName).distinct.size ==
        topic.choices.size) {
   Right(topic)
 } else {
   Left("There are duplicate choices!")
 }
```

Scala Parser Combinators

```scala
class ExampleParser extends RegexParsers {

  def parseDSL(input: String): ParseResult[Topic] =
    parseAll(topicParser, input)

  // ...
}
```

Scala Parser Combinators

```
class ExampleParser extends RegexParsers {

  def parseDSL(input: String):
      ExampleParser.this.ParseResult[Topic] =
    parseAll(topicParser, input)

  // ...
}
```

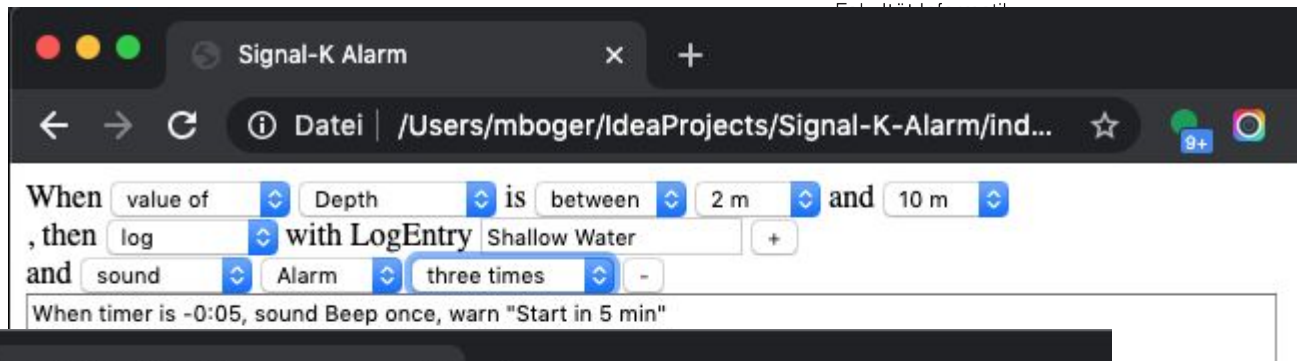
Alarm

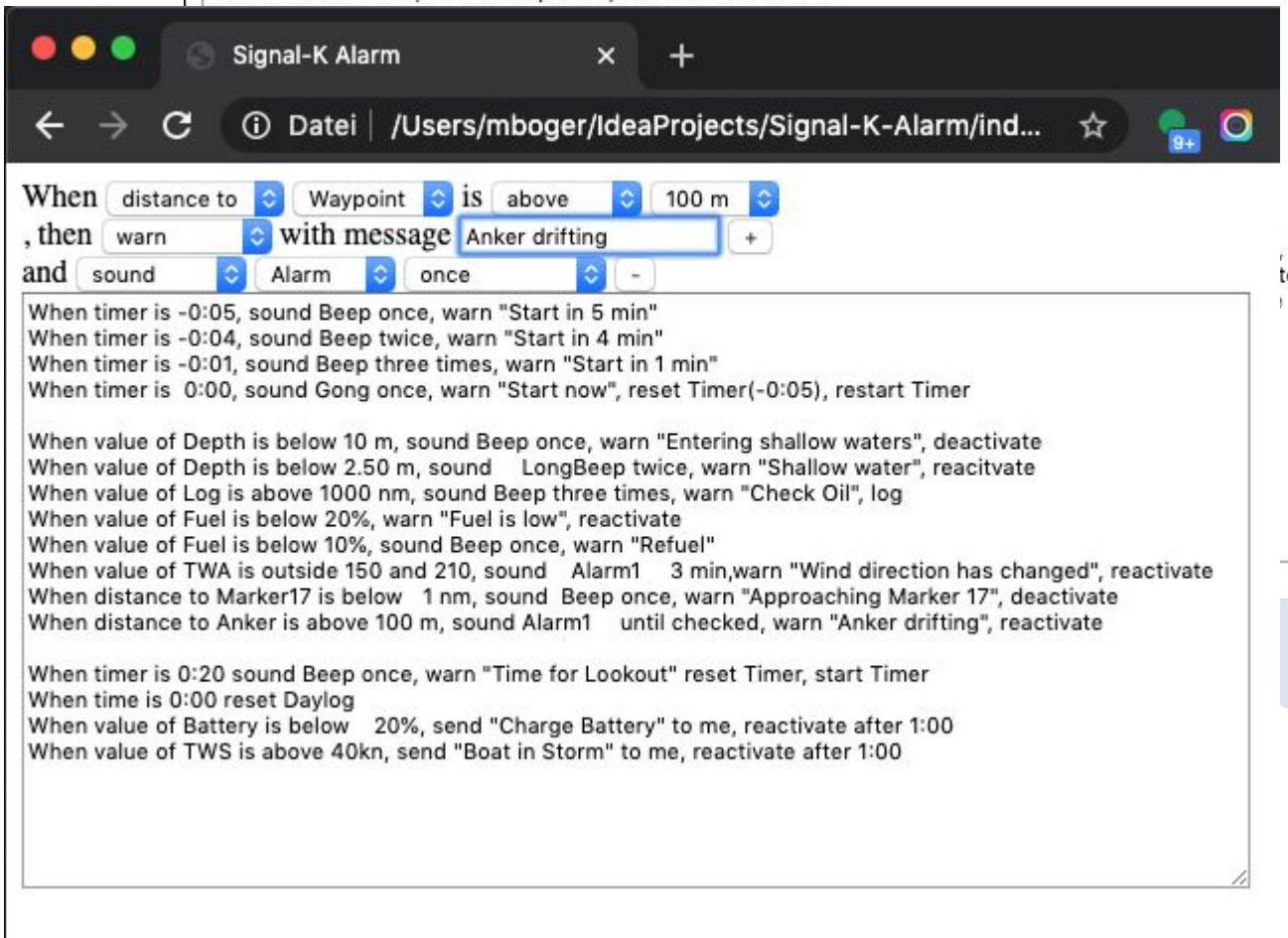
Sentence Builder

Textual DSL

Parser

Alarm API





A Rule Parser for Alarms

```
class RuleParser extends RegexParsers {


 def sign = "[-]".r ^^ { _ => -1 }
 def hour01 = ("([0-1])?[0-9]".r ~ ":") ^^ { case string ~ _ =>
Hours(string.toInt) }
 def hour2 = ("2[0-3]".r ~ ":") ^^ { case string ~ _ => Hours(string.toInt) }
 def hour = hour01 | hour2
 def minute = "[0-5][0-9]".r ^^ { string => Minutes(string.toInt) }
 def timeExp = hour ~ minute ^^ { case hours ~ minutes => hours + minutes }
 def timeClause = "time" ~ "is" ~ timeExp ^^ { case t ~ _ ~ time => new
Trigger[Time](t, TimeOperator[Time].is(time)(_)) }
 def timerClause = "timer" ~ "is" ~ opt(sign) ~ timeExp ^^ { case t ~ _ ~ sign ~
time => new Trigger[Time](t, TimeOperator[Time].is(sign.getOrElse(1) * time)(_)) }
```

https://github.com/markoboger/SignalkRuleParser/blob/master/src/main/scala/de/htwg/signalk/RuleParser.scala